

03.06.2024г.

0001
≡ лекция 16

Type casting

→ Искаме да преобразуваме от 1 тип в друг

1 вариант) `int x = 7;`
`double y = x;`

`int x = -1;`
`unsigned y = x;` // ще се отнегата пох на int
заради побитовите и стойн.

2 вариант) `Base`
`Der`

`Der* d = new D();`
`Base* b = d;` → `Base* b = (Base*)d`
C-style cast

1. Static-cast // шаблонна функция е

`Base* ptr = static_cast<Base*>((ptr));` → указател към Der

→ Използваме го само когато сме сигурни в типа
в който преобразуваме.

сигурен съм, че
преобразува в правилния
за указателя тип.

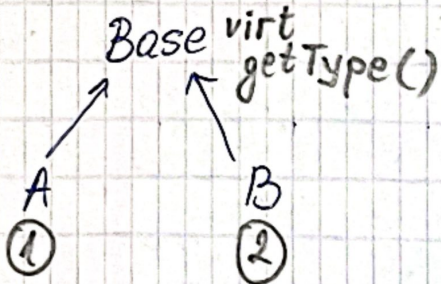
⚠ Не прави runtime check!!!
тоест, ако сбъркаме, то crash!

Base

Der
↑



→ upcasting



```
for (Base* ptr)
{
    if (ptr->getType() == 1)
    {
        A* obj = static_cast<A*>(ptr);
    }
}
```

→ downcasting!

Base* ptr = new A();

B* b = static_cast<B*>(ptr);

// това ще
crash-не
Ако не crash-не,
е undefined behavior

→ Използва се много в стари C библиотеки

```
for (void* ptr, callback)
{
    ...
    callback(ptr, ...);
}
```

не знаем какво е
може да сочи към
всичко, все едно е
универсална.

Не можем да го дефинираме, трябва да се
кастне, не можем да мести
указателя "


```

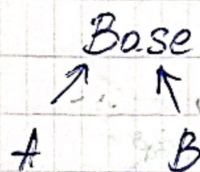
int x=0;
void *ptr = &x; → правам го да сози как int
f(ptr, f)(void * ptr)
{ int *data = static_cast<int*>(ptr);
}

```

II. dynamic_cast

→ Преобразување, но с runtime check
 (Ако работим со указатели и преобразување-
 то е неуспешно, то dyn_cast ќе врати
 nullptr)

- не се използват за upcasting
- използва се основно за down.cast,
 когато не знаеш какъв е тип



```

f(Base* ptr)
if (A* ptr = dynamic_cast<A*>(ptr))
{
    // логика
    за A, ако не е nullptr
}
else if (B* ptr = dyn_cast<B*>(ptr))
{
    // логика за B
}

```

ще се замени со aPtr
 и после проверка дали е nullptr

⚠ Проверката става през виртуалната таблица.
// там ще се пази типа.

⚠ Downcasting можем да правим, само ако имаме поне 1 виртуална таблица.

!(Base & ref)

A& refA = dynamic_cast<A&>(ref);

→ ако е неуспешен хвърля
std::badcast

III. Const_cast

→ Идеята е да манипулиране на константност на обекти, с които работим през указатели и референции.

⚠ НЕ ГО ИЗПОЛЗВАМЕ!

→ за да премахнем const трябва да сме сигурни, че при създаването си този обект не е бил константа.

int x = 10;

...
f(&x)

f(const int* ptr)

→ забранено да го махаме
{ int* n = const_cast<int*>(ptr);

(*n)++;

// можем вече да го промениме

Hebavungen use-case

```
const int x=10;
```

```
f(&x)
```

```
f(const int * ptr)
```

```
int * n = const_cast <int*>(ptr);
```

undefined behavior // защото constant.

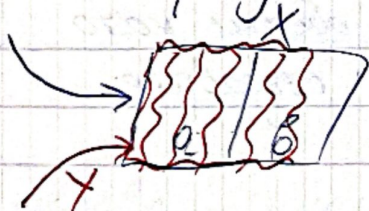
има специално
място в паметта и
не можем да го
местим

IV. reinterpret_cast

независими

```
class X ← ca → class Y  
int a;  
int b;  
char ch[5];
```

```
X* ptrx = new X(); // зареза памет  
Y* ptry = reinterpret_cast <Y*>(ptrx);
```



→ Реинтерпретация на памет.

```
int x=5;
```

```
f.write(reinterpret_cast <const char*>(&x));
```

// все така ще го
пишем за излизане заради

→ C style_cast

```
A* ptr = (B*) ptr2;
```

// това е послеробателност
от castове

- const_cast
- static_cast
- static_cast + const_cast
- reinterpret_cast
- ▼ - reinterpret_cast + const_cast

// рано или
късно някой
от кастовете
ще успее

Design Patterns

- Обобщени практики
↑
добри
- Решения на често възникващи проблеми
- три вида са:

① Creational patterns

- Осигуряват създаването на обекти като скриват логиката от тяхното създаване
- // пример Factory

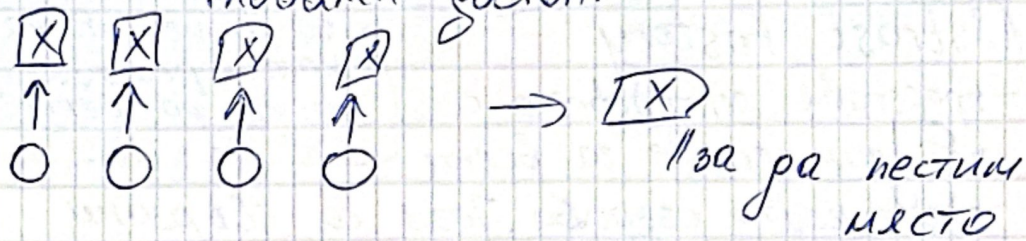
② Structural Patterns

- Начин на създаване на по-сложни ~~пре~~ обекти, използвайки инструменти като ~~на~~ наследяване и композиция

③ Behavioral patterns

- Коммуникация между обектите
- // ~~2~~ Visitor patterns

Singleton - осигурява само една инстанция на даден клас, към която има глобален достъп.



// String Pool - може да е singleton

// Правим Singleton Class

в private - конструктор и
в public: - деструктор
за да достигнем конструктора използваме
static функция и static променлива
изглеждаме copy const. ~~и може const~~
и оператор =

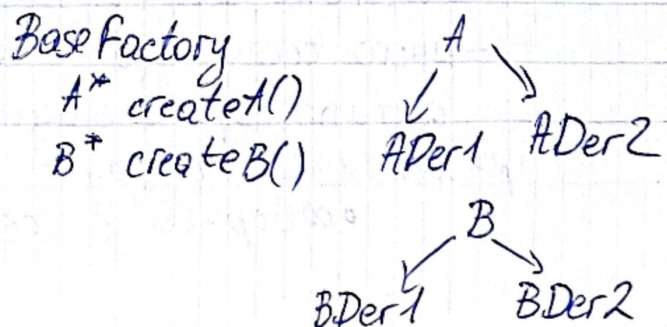
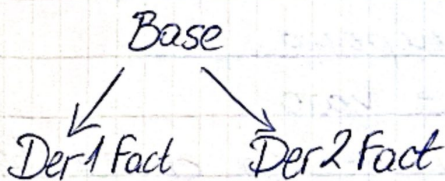
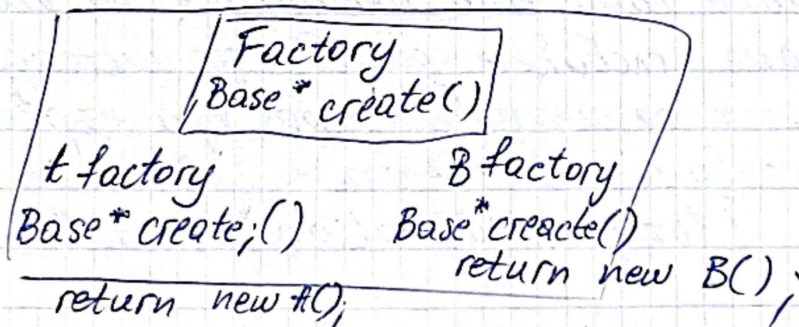
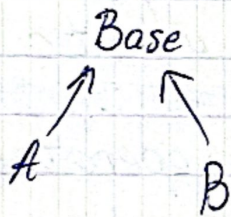
Плюсове - имаме само една инстанция на класа
- имаме глобален достъп до инстанцията
- обектът се инициализира при първото
достъпване на обекта
(lazy initialization) съставва ни го

Минуси - може да доведе до проблем с
инотонично програмиране
- много често тук се определят като
антипатърн, защото не може да се тества
при използване с друг код
• обвързваме се с конкретни
инстанции

- 1 Factory - статична функция (може в клас), която на база по даден аргумент връща instance
- 2 Factory Method - представя интерфейс с точно един create method. Всеки наследник презаписва имплемент. на create, която решава какъв обект да върне.
- 3 Abstract Factory - представя интерфейс с по един create за всеки обект, който създава. Тези са свързани логически

• Factory:

```
Base * create = 0;
AFactory::Factory()
A: Base
    new A();
```



Der1Factory

```
A* create(.) {  
    new ADer1  
}
```

```
B* create(.) {  
    new BDer1  
}
```

Плюсове ^{Method} for Factory, Abstract

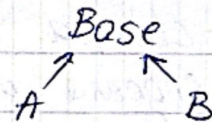
- генерално място за създаване на обект // 3
- по всяко време могат да добавят нов наследник на Base Factory // FM, AF
- // наследник на Base // 3

Минуси

- ново ниво на абстракция, т.е. по-сложен код / FM
- грижа за живота на Factory / FM, AF AF

Задатък за изпит:

n
x₁ x₂ x₃ ...
x_k % 2 = 0 new A
 ≠ 0 new B



четен файл с числа 9/5/4/.../

Singleton + Factory

→ Prototype - (Creation Patterns) → е полиморфна иерархия
- създаване на копие на обект без да се интересуваме какъв е типа
// clone

→ Composite pattern

- композиране на обекти в някаква дърбовурна структура

→ листа

→ междинни върхове

Файлова система

File virt getSize()

Folder
{ File**

TextFile

return size

$\sum f.getSize$

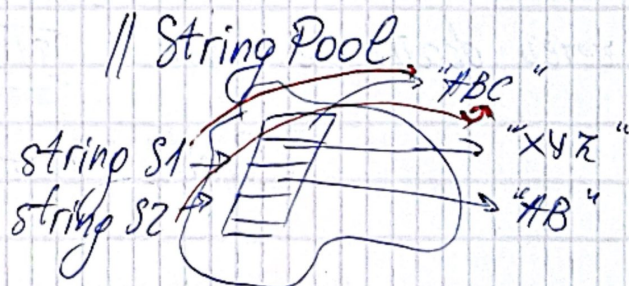
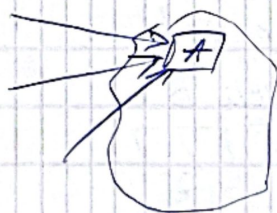
междинни
върхове

↑ листата

Пример: BooleanExprct.

→ Flyweight pattern - Structure Pattern

- събира повесе обекти в паметта, като споделя обектите или ресурсите.



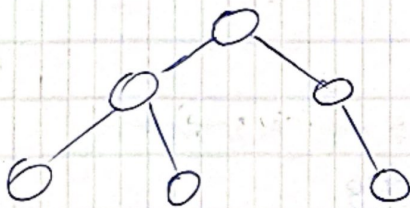
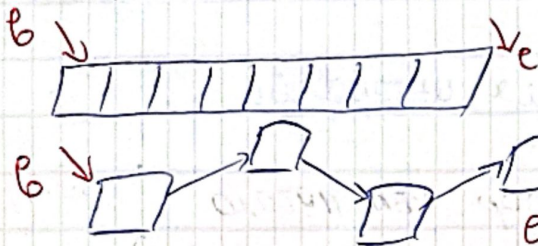
- подобрява бързодействието, ако създаването е тежка операция

Реализация : хеш/дърво

→ обектите са `Immutable` ! // не може да ги променяме

Iterator pattern

— начин за работа с колекции, без да се интересуваме каква е тя.



Итераторът е "указател" към конкретен елемент

Iterator `it`

`(*it)` — връща елемент

`OP++`

— отиди на следващия

`OP--`

— отиди на предишния

`OP*`

`OP!=`

`OP==`

// ако има тези `operators` то наричаме итератор


```

template <class IterType>
bool search (Iter begin, Iter end)
{ while (begin != end)
  { if (*begin == searched)
    return true;
    begin++;
  } return false;
}

```

Коллекцията трябва да има следния интерфейс

collection
 collIter begin() — връща итератор към началото
 collIter end() — връща итератор към края

Command pattern → за проектира се използва

→ програма, която прегледа заявки

> create ...

> find ...

> print ...

Command

virtual execute() = 0;

CreateCommand
 execute()
 override

PrintCommand
 execute()
 override

Command * current = CommandFactory();
 current → execute();

requests — обвиват се в обекти

→ може да ползвате команди като параметри

→ ~~delay~~ на команди

— (Behaviour Pattern)